BERTDAO PROTOCOL

TECHNICAL WHITEPAPER / VERSION 1.2

Transparent capital coordination for communities and grants.

A stablecoin-native protocol for stake-backed proposals, auditable decision-making, controlled treasury flows, and milestone-based release.

PUBLICATION
12 October 2026

CURRENT ENVIRONMENT
Arc Testnet / active development

PROTOCOL SCOPE
BERT V2 Global Funding Layer and BERT V3 Community Layer

CANONICAL SOURCE
github.com/Tenyokj/bert-core-arc

DOCUMENT STATUS

Read this before evaluating BERT.

Development-stage disclosure.

BERT is deployed and exercised on Arc Testnet. Testnet transactions are real testnet transactions, but test assets and testnet outcomes do not have monetary value. This paper describes the current implementation and intended architecture; it is not an offer, investment memorandum, promise of returns, security audit, or mainnet launch announcement.

Identity disclosure.

The public Arc Testnet dApp uses an explicitly labelled Demo verification path. It exercises the actual backend-attestation and onchain `PoPVerifierUpgradeable` flow, but it is not proof of personhood and must not be interpreted as Sybil resistance. The planned production policy is a World ID proof-of-personhood integration, subject to completion, review, and deployment readiness.

Abstract

BERTDAO Protocol is programmable, stablecoin-native infrastructure for coordinating proposals and capital. BERT V2.3 operates a global grant flow: target-backed proposal intake, immutable milestone plans, round-based refundable USDC pledges, viable-winner selection after a snapshotted fee, progressive reserve accounting, and winning-backer-governed milestone releases. BERT V3 is an isolated Community Layer: each Community has its own immutable governance Hub and USDC Treasury, local membership, validator review, binary and slate voting, and quorum-protected administrative actions. Defined V3 protocol inflows route into the existing V2 global reserve without merging Community execution balances or governance state.

The protocol is deliberately modular. Proposal metadata, vote lifecycle, token accounting, grant payout state, role authority, reputation, and progression each have a defined onchain owner. This reduces ambiguity for integrators and auditors: the interface is not the source of truth, and an offchain index does not replace a contract read when finality matters.

Contents

01 Problem and thesis
02 Design principles and scope
03 System architecture
04 BERT V2: global funding layer
05 BERT V3: Community Layer
06 Human-verification boundary

07 Security, trust, and operations
08 Data, indexing, and interfaces
09 Arc Testnet deployment
10 Mainnet readiness conditions
11 Appendix: addresses and glossary

01 / PROBLEM AND THESIS

Capital decisions should be inspectable, not trapped in spreadsheets and chat.

Grant programs and community treasuries often split a single decision across forms, informal moderation, spreadsheets, multisig messages, and offchain payout tracking. That fragmentation makes it difficult to answer basic questions: who proposed an allocation, what capital supported it, what was decided, where are the funds, and what evidence unlocked a payment?

The coordination problem

A useful treasury process needs more than a vote. It needs a way to resist low-cost spam, a lifecycle that cannot be rewritten casually, capital accounting that distinguishes deposits from obligations, and an execution process that does not release all funds before work is evidenced.

01 / INTAKE

Unpriced proposal spam

Without an economic commitment or review path, a public intake surface can become a queue of low-signal submissions.

02 / DECISION

Opaque capital signaling

A social poll does not explain how much capital was committed, which rules applied, or what happens on a failed result.

03 / EXECUTION

One-shot payout risk

A winning proposal is not evidence that all future work has been delivered. Treasury release needs its own accountable state machine.

BERT thesis

BERT treats proposal intake, selection, and payment as connected but distinct protocol stages. The V2 Global Funding Layer provides the shared grant and reserve machinery. The V3 Community Layer lets independent groups operate their own local governance and execution treasury while remaining connected to a defined global reserve route.

> **A proposal is not merely content. It is a lifecycle object with an author, a capital commitment, an outcome, and—when funded—an accountable release path.**

What BERT is and is not

BERT is	BERT is not
Onchain coordination infrastructure for USDC-denominated proposal and treasury flows.	A guarantee that any proposal is economically sound, lawful, or likely to succeed.
A modular protocol with separate state owners for ideas, rounds, funds, payouts, roles, and trust signals.	A replacement for human judgment, due diligence, or community accountability.
A testnet-stage system with real contract interactions and transparent documented limitations.	A production-ready, independently audited mainnet product today.
An integration surface for dApps, indexers, operators, and future Community deployments.	A custodial service or investment product.

Explicit state, explicit capital, explicit authority.

Design principles

Single source of truth by domain

Idea metadata belongs to `IdeaRegistryUpgradeable`; round state belongs to `VotingSystemUpgradeable`; treasury and reserve accounting belongs to `FundingPoolUpgradeable`; payout state belongs to `GrantManagerUpgradeable`; role assignments belong to `RolesRegistryUpgradeable`.

Economic actions are visible

Proposal deposits, vote commitments, execution balances, reserve routes, validator rewards, and refund liabilities are distinct accounting concepts. A displayed token balance must not be treated as generic free liquidity.

Local autonomy without silent pooling

A V3 Community has isolated state and custody. Its paired Treasury can only make defined protocol-reserve transfers through the authenticated V2 reserve route; a Community's membership stake and execution balance are not global reserve capital.

Human trust remains a boundary

Contracts can enforce configured thresholds and lifecycle rules, but backer diligence, Community validator quality, admin behavior, configuration choices, and future identity-provider operation remain social and operational trust assumptions.

Scope of this paper

This paper documents the public BERT V2 and V3 architecture as implemented in the core repository and Arc Testnet deployment manifest. It intentionally does not define token economics, a tradable token, speculative yield, NFT rewards, airdrops, legal compliance policy, or a final mainnet timetable.

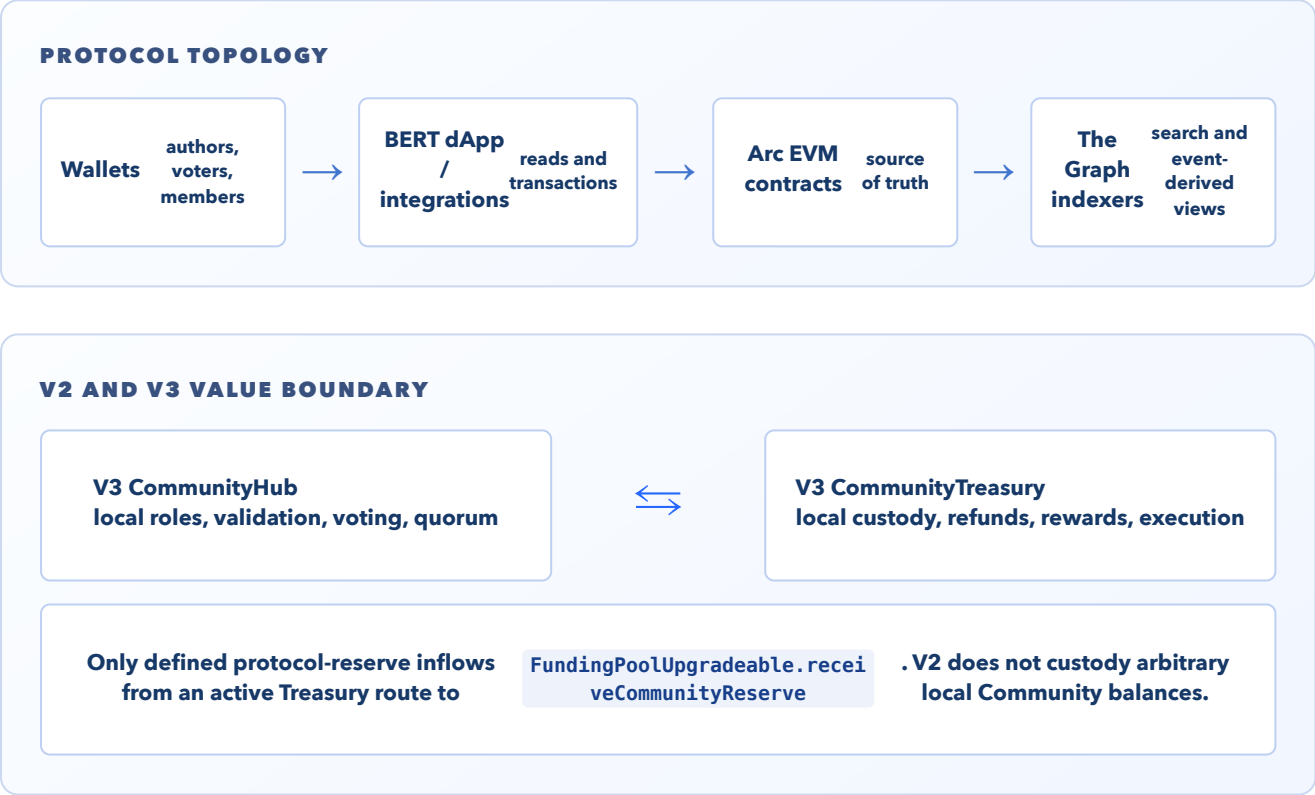
System actors

Actor	V2 participation	V3 participation
Builder / author	Submits a staked idea; may claim a winning grant and submit milestone proof.	As a Member, submits a bonded proposal after joining a Community.
Voter / member	Commits configured USDC stake in a funding round, subject to live gates.	Locks entry stake, then casts one stake-backed vote per binary proposal or slate round.
Backer / reviewer	A winning pledger governs V2 milestone escrow in proportion to its pledge; curator signals remain non-binding quality context.	Validator is a Community-local role that reviews Member proposals before they can be voted.
Administrator	Configures roles, dependencies, parameters, pauses, and upgrades within the current model.	Local Admin creates proposals and rounds, manages withdrawals, and uses immutable local quorum for protected actions.
Integrator / indexer	Reads contract state and V2 events; may use the V2 Subgraph for discovery.	Discovers Communities through Factory events and indexes Hub/Treasury event streams via the V3 Subgraph.

03 / SYSTEM ARCHITECTURE

Two connected layers, clear execution boundaries.

BERT V2 and V3 are complementary. V2 is the global funding and grant layer. V3 is the factory-created Community Layer. The connection is intentionally narrow: authenticated V3 Community Treasuries can route defined protocol inflows to the V2 reserve.



Contract deployment model

V2 core modules are upgradeable and interdependent. Their safety depends on storage compatibility, dependency wiring, role assignment, and controlled administration. V3 takes a different isolation approach: the shared `CommunityFactory` is upgradeable, while every deployed `CommunityHub` and `CommunityTreasury` is immutable for that Community. The Factory has no post-activation governance authority over a Community.

Source of truth and indexers

Contract storage and successful chain execution are authoritative. The Graph provides convenient event-derived discovery and UI views, but users and integrators should fall back to contract reads for security-sensitive state, especially role status, paused state, claimability, balances, and settlement status. The

public Arc RPC may prune historical logs; subgraphs reduce that read-path fragility for historical discovery but do not alter execution truth.

USDC denomination

Economic amounts are denominated in the configured USDC-compatible ERC-20's native units. Arc USDC is displayed with six decimals. Interfaces must not assume 18 decimals for user input, formatting, or allowance calculations.

04 / BERT V2

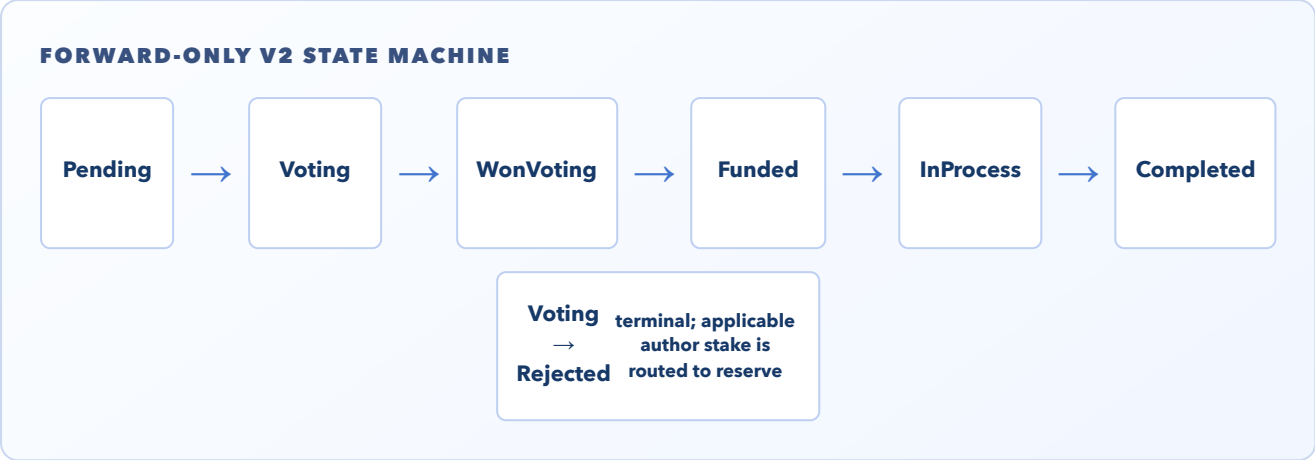
Global funding layer: proposals, rounds, reserve, and staged grants.

BERT V2 coordinates a protocol-wide grant lifecycle. Its modules keep intake, decision, accounting, payout, permissions, and social progression separate so no one contract becomes an ambiguous source for every kind of state.

V2 modules

Module	Primary responsibility	Important boundary
`IdeaRegistryUpgradeable`	Proposal metadata, author, lifecycle status, author stake, funding target and immutable milestone plan commitment.	Does not custody USDC; it instructs FundingPool through authorized paths.
`VotingSystemUpgradeable`	Sequential round formation, 14-day pledge windows, vote recording, viable winner selection and lazy progression claims.	Records policy and outcome; FundingPool performs token transfers/accounting.
`FundingPoolUpgradeable`	USDC custody and accounting for deposits, author stakes, votes, reserve, and distributions.	Accounting buckets are purpose-specific and cannot be treated as one free balance.
`GrantManagerUpgradeable`	Grant claims, backer milestone proof/votes, progressive fee vesting, expiry/cancellation and refund activation.	Cannot release the same stage twice; it relies on a valid V2.3 round and immutable winning cohort.
`RolesRegistryUpgradeable`	Protocol and user role authority.	Misgranted system roles are a critical trust and operational risk.
`ReputationSystemUpgradeable` and `VoterProgressionUpgradeable`	Outcome-linked reputation and earned user-role progression.	Reputation is not token value or voting weight.

V2 proposal lifecycle



1. Staked idea creation and plan commitment

An author calls `createFundingProposal` on the Idea Registry with non-empty title and description, optional link, an author bond at least equal to `authorMinStake`, and a non-zero `minimumNetFunding` target. While the idea remains `Pending`, the author must commit an immutable milestone plan URI and content hash. The package defines two milestones, acceptance criteria, expected evidence and the fixed 20/40/40 schedule. A V2.3 proposal without this commitment cannot enter a V2.3 funding round.

2. Round formation and pledging

An operator starts a round only when the next 30 sequential pending V2.3 ideas are eligible. The live Arc Testnet policy opens a 14-day pledge window. A backer can make one pledge in a round, cannot pledge for their own idea, must meet the current minimum stake, and is subject to the configured verified-human policy and per-pledge cap when enabled. Each pledge remains associated with that backer, round and selected idea.

3. Deterministic resolution

After the full window ends, any eligible caller may settle the round. The protocol evaluates ideas in rank order and selects the first one whose gross total less the snapshotted fee reaches its declared minimum net funding target and whose cohort contains at least three distinct backers. If no idea is viable, the round resolves with no winner and every recorded pledge is refundable by its original backer. Settlement does not loop through the cohort; a winning backer later claims its non-financial progression credit individually.

Capital is accounted for by purpose.

FundingPool accounting model

The V2 FundingPool distinguishes the following categories. These are accounting responsibilities, not a promise that all USDC represented by a broad UI total is withdrawable or distributable in the same way.

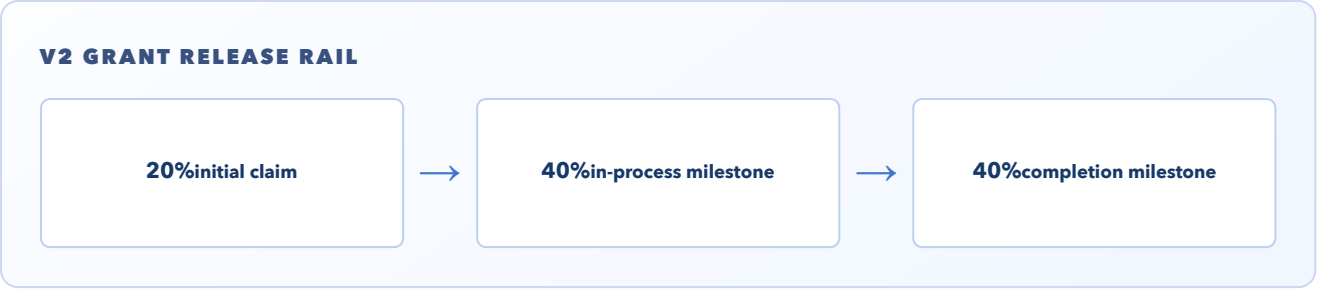
Bucket or mapping	Meaning	Typical origin
`donorBalances`	Record of direct contributor deposits.	`deposit(amount)`
`authorStakes`	Proposal-linked collateral, separate from a general donation.	Idea creation through the Registry
`poolByRoundAndIdea`	Capital committed to a particular idea in a particular round.	Votes through VotingSystem
`protocolReserve`	Capital tracked outside active distributable round balances.	Slashed/rejected stake, explicit reserve movements, authenticated V3 reserve inflows
`totalPoolBalance`	Live accounted pool value, reconciled with the token balance subject to reserve separation.	System accounting

Interpretation rule.

A valid integration must present reserve, active round capital, locked author stake, and already-released payout as separate concepts. Collapsing them into one "treasury available" number would misrepresent the protocol's obligations.

Grant claim and milestone releases

A winning author may call `claimGrant(roundId)` only after a viable round is ended, before the 14-day claim deadline and before the grant has been claimed. The contract determines the winning idea and reads its round-specific gross escrow. The current 5% round fee is held pending at settlement and vests only alongside capital actually released to the author. The grant is not released at once.



For each milestone, only the settled winning backer cohort may vote, with weight equal to each backer's recorded pledge. The author cannot vote. A release requires participation from at least 40% of total winning pledge weight, at least two-thirds approval among participating weight, and at least three verified-human approving backers. The review remains open for 14 days even if thresholds are reached early. No quorum receives one seven-day grace window; silence never approves a payment. A quorum failure after grace, a missed submission deadline, or a second rejected proof activates pro-rata refunds for the whole unreleased gross escrow. One corrected proof is allowed after a 48-hour cooldown.

Reputation and progression

V2 separates reputation from progression. Reputation records outcome-linked performance and is not a treasury balance or vote multiplier. Winning backers claim progression lazily after settlement, so an unlimited cohort does not make settlement unbounded. Curator and reviewer quality signals remain distinct from V2.3 milestone authority, which belongs to the winning pledge cohort.

V2 policy surfaces

Stake minima, round size, duration, pledge fee, minimum funding targets, vote caps, verified-human gating, trusted signer, and module dependencies are operational policy. Any future governance process must treat parameter changes as security- and UX-relevant changes, rehearse them on a staging environment, and record their effect in a changelog.

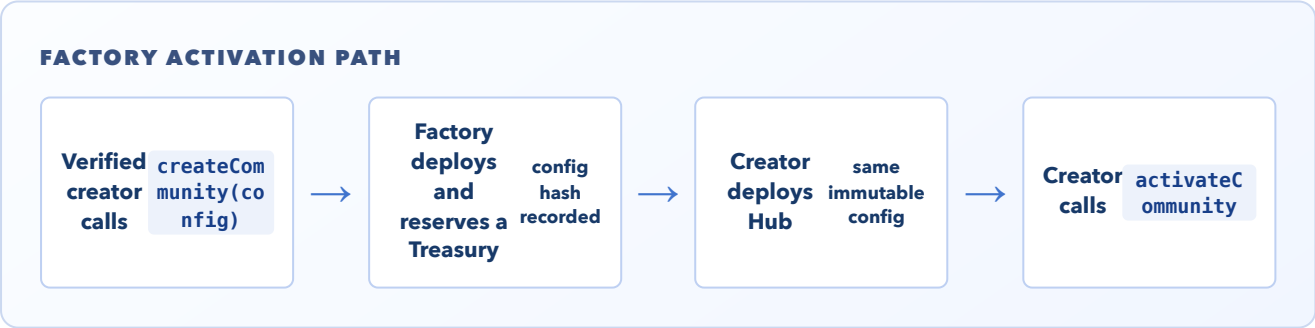
Community Layer: isolated governance with reviewable execution.

BERT V3.1 enables an organization or group to create its own stake-gated governance domain. Every active Community combines an immutable `CommunityHub`, a paired `CommunityTreasury`, and focused runtime modules registered and activated by the shared `CommunityFactory`. V3.0 is retained only as historical Arc Testnet state; new Communities use V3.1.

Why V3 is separate from V2

V2 organizes global funding rounds and grants. V3.1 is designed for local Community governance: a Community needs its own membership boundary, its own Admin and Validator set, its own proposal review, its own voting electorate, its own Treasury, and its own protected execution process. Mixing those local rules into the V2 global state would weaken isolation and make policy harder to reason about.

Deployment and activation



Factory activation verifies that the Hub reports the reserved Treasury, recorded creator, and exact configuration hash. The Treasury is then linked once to its Hub. This prevents activation of a Hub for a different creator, a different Treasury, or different immutable parameters. The Factory records Community IDs and exposes `isActiveCommunityTreasury`. FundingPool authenticates active Treasury reserve inflows through an explicit multi-Factory allowlist so V3.0 and V3.1 can coexist during testnet migration.

V3 contract	Role	Mutability
`CommunityFactory`	Shared registry, Treasury reservation, Hub activation, active-Treasury authentication.	Upgradeable Transparent proxy.
`CommunityTreasuryDeployer`	Deploys a pending Community Treasury for Factory.	Immutable deployment utility.
`CommunityHub`	Thin local coordinator, immutable configuration, active-time clock, and quorum-protected Admin actions.	Immutable per Community.
`Membership / Proposal / Voting modules`	Role exclusivity, proposal lanes, Validator review, binary/slate voting, winner snapshots and exit locks.	Immutable runtime modules per Community.
`CommunityMilestoneManager`	Evidence, supporter review, correction, failure and refund lifecycle for execution plans.	Immutable runtime module per Community.
`CommunityTreasury`	Local USDC custody, locks, settlements, rewards, execution-plan reserves, refunds, withdrawals, reserve route.	Immutable per Community.
`CommunityAdminActions`	Linked library used through the Hub for quorum-protected control-plane actions.	Linked immutable library code.

Membership and authority are Community-local.

Community roles

Role	Capabilities	Safety boundary
Creator	Reserves and activates the Community; must appear in the initial Admin set.	After activation, Factory gives no unilateral Community control to the Creator.
Admin	Creates Admin proposals and rounds; creates/approves execution withdrawals; creates and approves protected actions.	One Admin cannot unilaterally change roster, pause/archive, or cancel a withdrawal when the immutable quorum is greater than one.
Validator	Reviews Member proposals before they reach voting; may earn epoch rewards when activity thresholds are met.	Validator is not automatically assigned merely by reaching the proposal-point threshold.
Member	Creates Member proposals, votes, requests membership exit.	Must lock entry stake and cannot exit while governance obligations remain unresolved.

Role exclusivity and membership

An address may not simultaneously hold more than one active Community role state: Admin, Validator, or Member. A wallet joins by approving the paired Treasury and calling `joinCommunity`; the Treasury checks the shared human verifier and locks the immutable entry stake. Membership exit is explicit: request first, wait through the Community's cooldown, then finalize only after role conflicts, active proposals, and uncleared vote locks are gone.

Community-local roles are intentionally separate from V2 `RolesRegistryUpgradeable`, reputation, and progression. A V2 reviewer or curator has no automatic V3 authority. This avoids importing a global status into a local governance trust model without a Community decision.

Validator eligibility

A former Member can become eligible for nomination after accumulating the Community's immutable number of winning Member proposal points. That threshold is selected at creation and must be between 1 and 100. Eligibility is only a precondition: an Admin quorum still adds the Validator, and role exclusivity must be satisfied. The intent is to connect validator eligibility to demonstrated local outcomes without treating an automated metric as an automatic role grant.

Pause-aware time

V3 uses a Community clock that excludes paused time. Validation windows, voting windows, validator epochs, and protected-action expiry are measured against that clock. When archived, the clock freezes. This prevents a pause introduced during an incident from silently consuming deadlines that Members, Validators, or Admins could not act on.

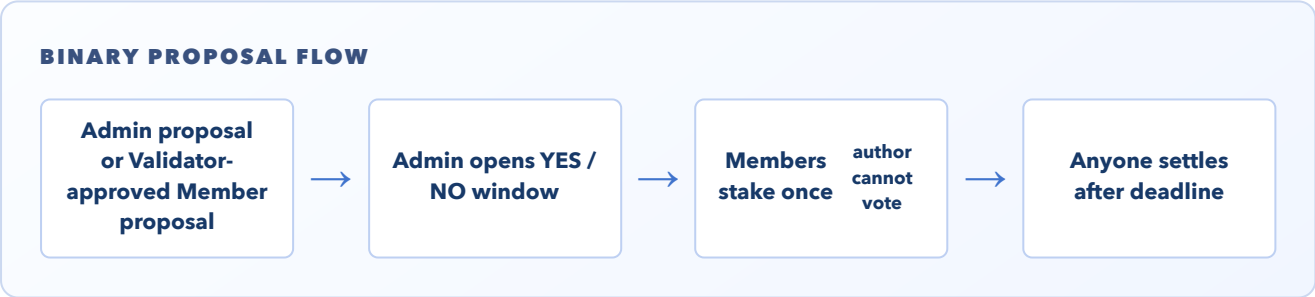
From proposal review to settlement.

Member proposals have a validation gate

Members create either binary or slate proposals. A Member proposal immediately locks the configured USDC anti-spam bond and enters `PendingValidation`. Validators cast one approval or rejection during the configured validation window. The proposal passes when approvals reach the immutable threshold; it fails early when the remaining possible approvals cannot reach that threshold; otherwise it can be finalized after the deadline using the approvals received.

On validator rejection, the Member proposal bond is routed to the global BERT reserve. If the proposal passes validation, its bond is returned when the later vote or slate round settles, whether it wins or fairly loses. The distinction is intentional: validation rejection is a moderation decision; a fairly considered democratic loss should not use the same economic penalty.

Binary proposals



Each Member chooses one side and commits an amount at least equal to the Community minimum and no greater than the protocol-wide cap of 10,000 USDC. YES must strictly exceed NO to win; a tie follows the NO/refund path. A proposal author cannot vote on their own proposal. The Hub records governance state; only the paired Treasury moves the corresponding USDC.

YES outcome

A winning Admin or Validator-approved Member proposal creates an immutable execution plan. The Treasury reserves the plan budget and releases 20% to the immutable Administration execution wallet at settlement. The remaining 80% is not a generic Admin withdrawal balance: it is released only as two 40% tranches after winning supporters review submitted evidence. A configured Member-origin validator reward share remains a separate Treasury obligation; actual immutable Community configuration is authoritative.

NO or tie outcome

YES stake and the configured NO-side rejection fee route to the global reserve. The remaining NO stake becomes a pull-based refund liability; each eligible NO voter must claim their own refund. The rejection fee is immutable per Community and cannot exceed 10% (1,000 basis points).

Slate rounds

Admins can open a homogeneous slate round containing approved Admin slate proposals or validator-approved Member slate proposals. A Member selects exactly one proposal and stakes once. The highest total wins; ties resolve deterministically in favor of the earliest proposal in the supplied round order. Slate rounds do not have a NO side. The selected proposal creates the same execution plan and supporter snapshot as a Binary YES winner; for Member-origin rounds, configured validator reward policy remains separate.

Community Treasury: custody follows validated state.

The Hub never custodies USDC. It validates lifecycle transitions and instructs its paired Treasury. The Treasury separates funds by obligation so membership stake, bonds, escrow, refunds, validator rewards, plan reserves, and general Treasury balances cannot be casually mixed.

V3 Treasury buckets

Bucket	What it protects	Can it fund execution directly?
`totalMembershipLocked`	Refundable entry stakes of active Members.	No.
`totalProposalBondLocked`	Member proposal bonds pending return or reserve route.	No.
`totalVoteEscrow` / `totalRoundVoteEscrow`	USDC committed to unresolved binary votes or slate rounds.	No.
`totalRefundLiability`	NO-side funds owed through pull-based refunds.	No.
`validatorRewardBalance`	Unclaimed, epoch-accounted validator rewards.	No, except defined rounding/no-active-validator return paths.
`milestoneReserveByPlan`	Unreleased budget of a winning proposal or slate candidate.	No; only the approved 20/40/40 plan path can release it.
`milestoneRefundByPlan`	Unreleased plan USDC owed pro rata to the winning-supporter snapshot after failure.	No; eligible supporters pull their own claim once.
`executionBalance`	Funds eligible for separately quorum-approved Community withdrawals.	Yes, subject to withdrawal reservation and Admin quorum; it is not a plan reserve.
`reservedExecutionBalance`	Execution funds already reserved by pending withdrawals.	No, until cancellation or request resolution.

Validator reward epochs

Validator rewards are measured in epochs. Member-proposal validation cases are counted, and an active-validator threshold determines whether a Validator is eligible. After the epoch and associated validation window close, rewards are finalized as equal claimable shares for active Validators. Rounding dust—or all rewards when no Validator is active—returns to execution balance. Claims are pull-based and protected against double claim.

Execution plans and withdrawals

For every settled winner, only wallets in the sealed winning-supporter snapshot can review a milestone proof. Review uses support weight, not one-wallet-one-vote: at least 40% of supporter weight must participate, two-thirds of participating weight must approve, and the plan also needs at least `max(3, ceil(3% of supporter wallets))` approving wallets. The review window stays open for the immutable Community duration even after thresholds are reached. One rejected proof may be corrected after cooldown. A second rejection, proof expiry, or no quorum after grace fails the plan; only the unreleased amount becomes a pull refund for winning supporters. Silence never releases a tranche.

Separately, a current Admin can create a withdrawal request only from available `executionBalance`, not from plan reserve. The request records recipient, amount, reason, metadata URI, and the creator's first approval; it reserves the amount immediately. Other current Admins approve once each. The Treasury transfers only after the Hub's immutable Admin threshold is met. A pending withdrawal can be cancelled only through the separate quorum-protected Admin action process, preventing one Admin from unilaterally undoing a reserved action.

Archive safety

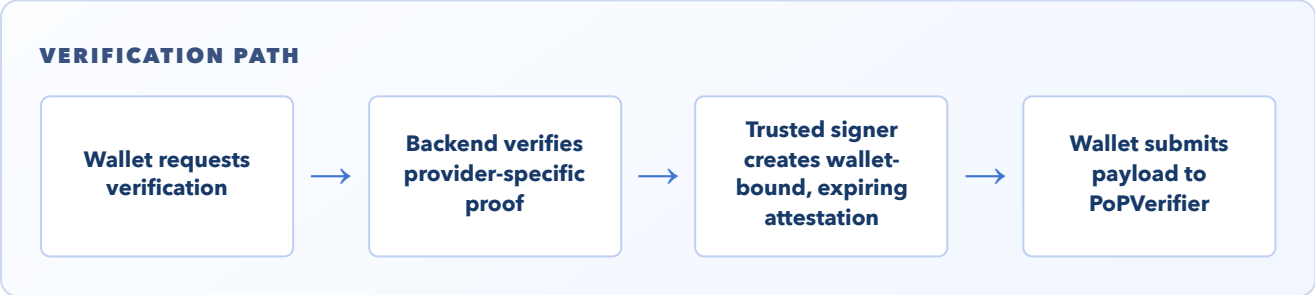
Archive is a protected action. Community lifecycle changes, roster changes, pause/resume, and withdrawal cancellation use the linked `CommunityAdminActions` library through the Hub. The requester contributes the first approval; the immutable threshold is evaluated at execution time; requests expire after seven active Community days; paused time does not count. An Admin handover intentionally takes two actions: add the replacement, then remove the former Admin.

06 / HUMAN VERIFICATION

Provider-agnostic onchain gate, explicit offchain trust boundary.

BERT does not embed a vendor SDK in its Solidity core. Instead, `PoPVerifierUpgradeable` records signed verification attestations and exposes a narrow `isVerifiedHuman(account)` read. Protected contracts decide whether to require that read for a configured action.

Attestation model



The verifier validates the trusted signer signature, the wallet binding, nonce and expiry, then stores the active record. The contract is intentionally provider-agnostic: policy can evolve without coupling every protocol module to external identity-provider code. The tradeoff is explicit: the backend, signer key management, proof-validation implementation, and provider policy become a meaningful trust boundary.

Protected actions

In V2, the live policy can require verified human status for voting and idea creation. In V3, the Factory requires a verified creator to reserve a Community and the Community Treasury requires a verified wallet before locking entry stake and activating Membership. Read-only calls and asset-recovery flows are not indiscriminately gated, so a verification policy does not strand an existing user's funds.

Testnet and production are intentionally different

Environment	Current path	Security claim
Arc Testnet	Demo verification with provider ID `BERT_TESTNET_DEMO`; real backend signature and onchain verifier flow.	No unique-human or Sybil-resistance claim.
Planned mainnet	World ID proof-of-personhood policy, backend proof verification, nullifier-to-wallet binding, and signed onchain attestation.	Not active until migration, independent review, operational controls, and production deployment are completed.

Security interpretation.

A valid onchain Demo PoP record proves only that the current testnet signer issued the configured demo attestation. It does not prove a unique human. Any team evaluating BERT must distinguish functional test coverage of the verifier path from production identity assurance.

07 / SECURITY AND OPERATIONS

Correct contracts still require correct operations.

BERT's primary risks are not limited to conventional Solidity bugs. The protocol is multi-contract and upgradeable: security also depends on authority, configuration, dependency wiring, identity-signing operations, indexer behavior, and the social quality of backers, Community validators and administrators.

Security controls in the current design

- ✓ Explicit role gates for treasury, lifecycle, reputation, and cross-contract actions.
- ✓ Forward-only V2 idea lifecycle and stage-aware payout records to resist duplicate or skipped grant release.
- ✓ Separated accounting buckets for V2 reserve and V3 local obligations.
- ✓ `SafeERC20` transfers and reentrancy protection around sensitive token flows.
- ✓ Pause controls on critical V2 modules, and pause-aware V3 Community time.
- ✓ Immutable V3 per-Community configuration, paired Hub/Treasury activation checks, and Factory authentication for reserve inflows.
- ✓ Admin quorum for V3 roster changes, pause/resume, archive, and withdrawal cancellation.
- ✓ Tests covering core V2 flows, V3 Factory/Hub/Treasury flows, edge cases, upgrade behavior, and non-standard token scenarios.

Material trust assumptions

Boundary	Why it matters	Required production control
Upgrade and admin authority	Incorrect upgrades or role assignments can compromise availability or integrity even when storage is preserved.	Multisig ownership, timelock/policy where appropriate, rehearsed upgrades, public change records.
Verification backend and signer	A compromised signer or faulty provider verification could issue improper access attestations.	Key isolation, rotation/revocation procedure, production World ID validation, monitoring, incident plan.
Backers and Validators	Winning V2 backers decide milestone releases; V3 Validators influence Community proposal admission.	Clear evidence packages, threshold disclosure, direct-read UI, notification policy, documented Community selection and conflict management.
Configuration and dependency wiring	A correct contract at a wrong address can break live flows or route value incorrectly.	Deployment manifest, post-deploy smoke tests, address verification, role inventory.
External RPC and subgraph availability	UI discovery may be incomplete or delayed even when chain state is valid.	Fallback RPC/indexer strategy, freshness indicators, direct contract read fallback.

Upgrade discipline

Storage compatibility is necessary but insufficient. New storage must be appended; inherited layout and storage gaps must be preserved; proxy target and admin must be verified. After any upgrade, operators must validate live implementation address, dependency addresses, role assignments, human-verification configuration, idea creation, voting, grant claimability, and the relevant V3 reserve boundary. A rollback is a controlled re-upgrade, not an automatic undo mechanism.

Disclosure and incident response

Public source repositories use private GitHub vulnerability reporting for security disclosures. A production runbook must preserve the ability to pause the minimum necessary V2 modules, capture state and transaction evidence, rotate affected credentials, patch and rehearse remediation, and publish an incident summary after safe resolution.

Audit status.

The current testnet codebase has automated and manual test coverage, static-analysis work, public source review, and a disclosure path. It has not yet completed the independent smart-contract audit required before any mainnet capital deployment. No reader should infer otherwise.

Contracts execute. Indexers make history usable.

Integration rule

An integration should use direct contract reads for final, security-sensitive data and use indexed events for discovery, activity history, search, and performant lists. The intended client model is not to scan all logs from genesis through a public RPC on every page load; such requests can fail on pruned RPC history.

V2 reads and events

Need	Authoritative contract surface	Useful indexed material
Idea detail and status	`IdeaRegistryUpgradeable` idea/status reads	Idea creation, status, vote, review events
Round detail and result	`VotingSystemUpgradeable.getRoundInfo`	Round started, vote cast, round ended events
Claimability and payout	`GrantManagerUpgradeable.canClaimGrant` and payout reads	Grant and milestone events
Reserve and accounting	`FundingPoolUpgradeable` state reads	Deposit, reserve, distribution events

V3 discovery model

The V3 Subgraph indexes the Factory and uses Community Hub event templates so newly activated Communities can become discoverable without hardcoding each Hub address into the frontend. This supports Community directories, role rosters reconstructed from role events, proposal history, rounds, Treasury activity, and lifecycle displays. Contract reads remain necessary where a current record decides whether a write action may proceed.

Developer safeguards

- Format Arc USDC with six decimals; do not assume an 18-decimal token.
- Distinguish a proxy address from an implementation address; users interact with the proxy/current contract address.
- Preflight allowance, current verification, pause state, role, timing, and status before asking a wallet to sign.

- After a transaction, wait for receipt and refresh authoritative contract state. A subgraph can lag behind chain finality.
- Do not make client-side constants the final policy authority; read configuration from the relevant contract.

Public resources

Resource	URL
BERT dApp	https://bertdao.vercel.app
Developer documentation	https://bertdao-docs.vercel.app
Core contracts	github.com/Tenyokj/bert-core-arc
V2 Subgraph	thegraph.com/studio/subgraph/bert-arc-testnet
V3 Subgraph	thegraph.com/studio/subgraph/bert-v-3-arc-testnet

Public deployment registry and current boundary.

The following addresses identify the active Arc Testnet deployment as of the V2.3 release on 7 October 2026. They are not mainnet addresses and must never be reused as assumptions for a future production deployment. The listed V3 addresses are V3.0 historical infrastructure; V3.1 is a separate modular release and its address manifest is published only after its reviewed testnet deployment.

Network

Network	Arc Testnet
Chain ID	`5042002`
USDC-compatible token	`0x3600000000000000000000000000000000000000`
V3.0 Factory deployment block	`61277385`

V2.3 release anchors

Action	Transaction
IdeaRegistry proxy upgrade	`0x25e04b6ab0565ce090536473922d30b3b9eddbb5bacb508b954093f670f835cf`
VotingSystem proxy upgrade	`0x27ee898d0856b0e5ee473040cd209282c75612f70da236ef04732518f4a7c461`
FundingPool proxy upgrade	`0x594bc919edfb04a2acc0c50cf3a12f755b18f0fc4d7e59c61e90e91507a6a20e`
GrantManager proxy upgrade	`0x69835e5aea756ff89b0257ce1a27a21d6465ec353ca643c5b0d72e261066b5f8`
Legacy queue retirement	`0x7bcf54e4ec97e178e1dfdbfc5e533d52cda049a914762de2d1a295b201c5aabe`

Shared and V2 proxy addresses

Contract	Proxy address
RolesRegistryUpgradeable	`0xf15c6213666EB4c09f594567DEf1345E2899BCC7`
PoPVerifierUpgradeable	`0xb8E2CB14F99E1a17C3Eee5863272A7F6dAF3C62b`
IdeaRegistryUpgradeable	`0xE6563acfdc194Ba3861EfD928Bd8B1B33a5D29d5`
FundingPoolUpgradeable	`0x5eDdccd772a9E0F079cC6646b529C20e3D46e585`
VotingSystemUpgradeable	`0xD008fC96902A9680FF77305881Ad6C6C075e7a68`
GrantManagerUpgradeable	`0xA110baB4562d59d8bb7eDA4a05E53F40c678ccB2`
VoterProgressionUpgradeable	`0xF6EB65957bb5e363FCD6B84AAdf76E41Ba754E14`
ReputationSystemUpgradeable	`0x3594C46983460733F6470f7b61De2f3bB2918a81`

Historical V3.0 infrastructure addresses

Contract	Address
CommunityFactory proxy	`0x811fFb3B53d43e608Ee4dF854aa1BdC471F37589`
CommunityTreasuryDeployer	`0x84594a42E59789C687e9397a994D71CFfa522409`
CommunityAdminActions library	`0x5EA79A8a98f588aa342f53baCBC8729A6Ea1177E`

Operational note.

The deployment manifest is the canonical repository record for addresses and upgrade transactions. A UI, whitepaper, or social post may become stale; operators and integrators must verify against the latest manifest and chain state before acting. V3.1 must be allowlisted in FundingPool alongside, not instead of, V3.0 before its first Community can route protocol reserve.

Mainnet is a gate, not a date.

The correct launch criterion is not whether BERT can be deployed. The code can be deployed today; a responsible production release requires evidence that its security, operations, identity policy, economics, and user flows are ready for assets with real value.

Required gates before mainnet capital

Gate	Minimum evidence
Independent smart-contract audit	Defined scope covering V2, V3, linked library, upgrade paths, reserve routing, identity verifier, and deployment configuration; findings remediated or explicitly accepted.
Testing and formalized invariants	Expanded fuzz/invariant suite for accounting, role exclusivity, settlement-once, refund liability, reward claims, lifecycle, and upgrade storage/dependency safety.
Production identity policy	World ID v4 migration/integration, nullifier and wallet-binding policy, backend secret isolation, monitoring, revocation/rotation runbook, privacy review.
Governance authority	Multisig ownership for upgrade/admin/pause powers, access inventory, signer policy, emergency process, and no single-wallet production control.
Operations and infrastructure	Production RPC fallback, production indexers, monitoring, alerting, backups, status page, incident communication policy.
Economic policy	Publicly justified limits, stakes, bonds, fees, reward shares, reserve policy, payout policy, and value-at-risk caps.
Usability evidence	Observed testnet sessions, design partners using end-to-end flows, resolved critical UX failures, mobile/accessibility review.
Disclosure and support	Security policy, bug bounty or responsible disclosure process, terms/risk disclosure, upgrade disclosure, and clear user support routes.

Testnet work that matters

Before a mainnet release, BERT should recruit real testnet users and small design partners rather than optimize for passive social metrics. The useful evidence is completed end-to-end flows, reproducible bugs, failed assumptions, data about user confusion, validator/admin workload, and documented operating procedures. This is also the right environment to rework the UI with an experienced frontend engineer.

Community and contributor model

Core contracts, backend, frontend, and documentation are public under GPL-3.0. Public visibility does not grant direct push rights. Contributors work through forks and pull requests; maintainers retain merge and deployment control. Security reports should remain private until a patch and coordinated disclosure are ready.

No implied launch promise.

This whitepaper intentionally does not name a mainnet date, token, yield, reward, or valuation. Any such statement should be made only after the relevant technical, legal, economic, and operational decisions are finalized.

Protocol invariants and glossary.

Selected invariants

Area	Invariant
V2 state ownership	Ideas, rounds, treasury/reserve accounting, payouts, roles, reputation, and progression each have a designated contract owner.
V2 lifecycle	An idea enters through `Pending`; only valid round resolution or grant transitions move it forward; rejected and completed states are terminal.
V2 pledges	A pledge remains attached to its selected idea; losing pledge capital is never redirected to a different winner.
V2 payouts	A tranche must not release twice; V2.3 milestone release requires 40% pledge-weight quorum, two-thirds participating approval and three approving backers.
V2 cancellations	Silence cannot approve a payment. Unreleased gross escrow becomes pro-rata refundable after the fixed failure paths.
V3 role state	An address cannot hold more than one active Community state among Admin, Validator, and Member.
V3 settlement	A binary proposal or slate round settles once; a winner receives an immutable supporter snapshot and one 20/40/40 execution-plan lifecycle.
V3 custody	Membership lock, bond lock, escrow, refund liability, validator rewards, generic execution balance, and per-plan reserve are separately accounted and not interchangeable.
V3 authority	No external wallet can invoke linked Admin action implementation directly; Hub rechecks caller and lifecycle rules at execution.
V3 reserve routing	V2 FundingPool accepts Community reserve inflows only from a Treasury active in an explicitly approved Factory; V3.0 and V3.1 Factory approvals coexist.

Glossary

Term	Meaning in BERT
Active Community	A Factory-recorded Community whose reserved Treasury has been linked to a validated Hub.
Admin quorum	The immutable number of Community Admin approvals required to execute protected control-plane actions or a Treasury withdrawal.
Author stake / proposal bond	USDC collateral locked when a V2 idea or V3 Member proposal is created; the exact lifecycle depends on its layer and outcome.
Community clock	V3 timestamp calculation that excludes paused time and freezes after archive.
Execution plan	V3.1 winner budget with a 20% initial release and two supporter-reviewed 40% releases. The Administration execution wallet has no generic Treasury withdrawal right.
Execution balance	V3 Treasury balance eligible for separately quorum-approved execution withdrawals after respecting reservations; it excludes per-plan milestone reserve.
Global reserve	V2 FundingPool accounting bucket for protocol-held value, including defined V3 reserve inflows; it is distinct from a V3 Community execution balance.
Winning pledge cohort	The immutable set of wallets that pledged to the settled V2 winner; only this cohort may govern its milestone escrow.
Milestone plan	V2.3 immutable author commitment or V3.1 winner execution plan. Both require evidence and explicit participant review; their electorates are layer-specific.
Human verifier	Onchain contract that reports whether a wallet has an active trusted verification record.
Slate round	V3 round where each Member selects one proposal from an ordered homogeneous slate; highest stake wins and order resolves a tie.
Validator reward epoch	V3 period measuring validation participation and determining equal pull-based rewards for active Validators.

Further reading

- BERT Core repository: contracts, deployment scripts, tests, manifests, and security documentation.
- BERT Frontend repository: Next.js dApp and user-facing integration.
- BERT Backend repository: verification and operational service surface.
- BERT Developers: evolving developer reference and Arc Testnet guidance.
- Private vulnerability reporting: coordinated disclosure path.

Disclaimer

This document is provided for technical and informational purposes only. It does not constitute financial, legal, tax, investment, or security advice. The BERT protocol is open-source software under GPL-3.0 and is provided without warranty. Contract interactions

can fail, code may contain undiscovered vulnerabilities, testnet behavior may differ from a future production environment, and smart-contract use involves risk. Verify current deployment artifacts and chain state independently before integrating or transacting.